

Import Wizard First - Product Specification

1. Purpose

Define a product focused on transforming unstructured and structured inputs into high-quality user journey and business process maps while reusing existing diagram editors instead of rebuilding them.

2. Product Strategy

Core Principle

- ****Do not build a new editor.****
- Build an ****Import-to-Map Intelligence Layer**** that outputs to existing editors/renderers.

Value Focus

- Automated extraction from raw content (upload + pasted text)
- Normalization into a canonical process model
- Human review workflow for confidence, ambiguity, and evidence
- Export to multiple diagram standards from one model

3. Target Users

- Operations and process excellence teams
- Product managers and service designers
- Business analysts and consultants
- Technical documentation owners

4. Problem Statement

Users have process information spread across SOPs, meeting notes, policies, tickets, spreadsheets, and free text. Existing editors are good at drawing but poor at extracting structure from messy documents. Manual conversion is slow, inconsistent, and hard to validate.

5. Goals

- Reduce time from document intake to first usable map.
- Increase semantic quality of generated process/user journey maps.
- Provide traceability from every generated node back to source evidence.
- Support iterative refinement before opening in downstream editors.

6. Non-Goals

- Building a brand-new full-featured diagram editor.

- Replacing enterprise BPM suites in v1.
- Real-time collaborative editing in v1.

7. System Architecture

7.1 High-Level Components

1. **Input Ingestion**

- File upload: PDF, DOCX, TXT, CSV, JSON, Markdown
- Paste text mode

2. **Extraction Engine**

- Document parsing and segmentation
- Entity and step extraction (actors, actions, systems, decisions, artifacts)

3. **Normalization Engine**

- Deduplication, ordering, merge synonyms
- Decision path shaping and handoff detection

4. **Canonical Model Store**

- Internal JSON schema (single source of truth)

5. **Review & Validation UI**

- Confidence and ambiguity queue
- Source evidence preview per generated item

6. **Visual QA Comparator**

- Side-by-side document map overview and imported process map
- Node, connector, cluster, theme, and title comparison metrics
- Mismatch heatmap and drill-down evidence links

7. **Compiler Layer**

- Mermaid output
- BPMN XML output

8. **Editor Integration Layer**

- Mermaid Live Editor embedding/launch
- BPMN editor integration (e.g., bpmn-js)

7.2 Canonical Model (Required Fields)

- ``process.id``, ``process.name``, ``process.type`` (``user_journey`` | ``business_process``)
- ``actors[]`` with role tags
- ``steps[]`` with:
 - unique id
 - actor reference
 - action verb + object

- optional system/touchpoint
- preconditions/postconditions
- estimated sequence index
- confidence score
- evidence references
- `decisions[]` with branches
- `handoffs[]` between actors/systems
- `artifacts[]` (documents/data objects)
- `exceptions[]` and rework loops

8. Functional Requirements

1. Users can upload one or more supported files and/or paste raw text.
2. System extracts candidate process/journey elements within a single guided run.
3. Each generated element includes evidence references to source snippets.
4. Users can approve, edit, merge, or reject generated items before final map generation.
5. System exports to both Mermaid and BPMN from the same canonical model.
6. Users can open output in integrated editor contexts for final formatting.
7. System stores session state for resume/re-edit.
8. System provides a Visual QA mode that compares:
 - source document map nodes vs imported process nodes
 - source connectors vs generated connectors
 - source clusters/themes/titles vs generated groupings/labels
9. Visual QA mode provides confidence and mismatch diagnostics per compared object.
10. Users can accept source-aligned corrections directly from the QA panel.

9. Non-Functional Requirements

- **Performance:** Initial extraction response starts quickly and progressive results stream by section.
- **Reliability:** Parsing/extraction failures isolate per document section, not full-run failure.
- **Security:** Input documents handled with strict temporary storage policy and clear retention controls.
- **Auditability:** Every generated node can show source evidence and transformation steps.
- **Extensibility:** New output formats can be added without changing extraction core.

10. Key UX Flows

Flow A: Quick Import to Draft Map

1. Select map type.
2. Upload files or paste text.

3. Run extraction.
4. Resolve ambiguities.
5. Generate draft map.
6. Open in editor.

Flow B: Multi-Document Reconciliation

1. Upload multiple process documents.
2. Compare conflicting steps and owners.
3. Resolve conflicts in review queue.
4. Generate consolidated process map.

Flow C: Iterative Refinement

1. Edit extracted model.
2. Re-run normalization only (without full re-parse).
3. Regenerate Mermaid/BPMN outputs.

Flow D: Visual QA Reconciliation

1. Load source-side document map overview.
2. Render imported process map from canonical model.
3. Run structural comparison for nodes/connectors/clusters/themes/titles.
4. Resolve mismatch suggestions.
5. Re-run comparison and approve.

11. Screen Requirements

- ****S1: Start/Map Type Selection****
- ****S2: Input Upload + Paste****
- ****S3: Extraction Progress****
- ****S4: Evidence & Ambiguity Review****
- ****S5: Visual QA Compare (Document Map vs Imported Map)****
- ****S6: Structure Editor (canonical model grid/tree)****
- ****S7: Generated Diagram + Editor Launcher****
- ****S8: Export + Share****

12. Editor Integration Recommendation

Primary

- ****BPMN editor integration**** for formal business process outputs.

Secondary

- ****Mermaid integration**** for lightweight sharing, docs, and technical teams.

Decision Rationale

- BPMN offers stronger process semantics and governance support.
- Mermaid offers fast text-first iteration and portability in documentation.
- Canonical model avoids lock-in and enables both.

13. Acceptance Criteria (MVP)

- Import from TXT, DOCX, CSV and pasted text.
- At least 80% of clear linear steps extracted correctly in benchmark set.
- Evidence links visible for 100% of generated nodes.
- Ambiguity queue supports approve/edit/reject/merge.
- Mermaid and BPMN export both available from same reviewed model.
- End-to-end run from upload to rendered map without manual JSON editing.
- Visual QA comparator reports node and connector alignment scores.
- Users can resolve mismatches from QA panel and regenerate maps without restarting import.

14. Metrics

- Time from upload to first draft map.
- Manual correction count per imported process.
- Percent of steps with confidence above threshold.
- Export success rate (Mermaid/BPMN).
- User completion rate of wizard runs.
- Node alignment precision/recall vs source document map.
- Connector alignment precision/recall vs source document map.
- Cluster/theme/title match rates after QA pass.

15. Risks and Mitigations

- ****Noisy source data**** -> Use chunking + evidence citations + ambiguity queue.
- ****Conflicting process versions**** -> Add conflict detection and reconciliation UX.
- ****Over-reliance on model output**** -> Require human approval gate before final export.
- ****Editor mismatch across formats**** -> Keep canonical schema stable and compilers versioned.

16. Delivery Plan (Technical)

1. Canonical schema + ingestion adapters
2. Extraction + normalization pipeline
3. Review/approval UI

4. Mermaid/BPMN compilers
5. Editor integration and export hardening
6. Benchmarking and quality tuning

17. Visual QA Input and Output Specification

17.1 Input Sources

- ****Primary source:**** Import wizard extracted canonical model.
- ****Reference source:**** Document map overview artifact (HTML/JSON/SVG metadata).

17.2 Comparison Objects

- Nodes (activities, decisions, milestones)
- Connectors (sequence, branch, merge, handoff)
- Clusters/swimlanes and theme labels
- Map titles/subtitles and major section headers

17.3 QA Output Panel

- Alignment scorecards (nodes/connectors/clusters/themes/titles)
- Missing/extra/conflicting object lists
- Suggested remaps (rename, merge, re-link)
- Evidence links to source snippets and source-map anchors

17.4 Integration Note for Local File Inputs

- The sample path provided by stakeholders (for example a local `file:///...html``) must be loaded by the client/browser picker at runtime.
- Backend services should not assume direct access to end-user local file paths.